EARTHLY LUNAR

Lunar on AWS EKS — Infrastructure Cost Analysis

Scenario: GitHub organization with 200 repositories, ~20 collectors and ~30 policies configured, running on customer-managed AWS EKS.

This document provides rough cost estimates based on representative assumptions. Actual costs depend on repository activity, collector and policy configuration, instance sizing, and AWS region pricing.

Summary

Based on the assumptions outlined in this document, running Lunar on AWS EKS for a 200-repository GitHub organization may cost in the range of **$300–550/month**. Actual costs will vary depending on repository activity levels, the number and type of collectors and policies configured, instance sizing choices, and AWS region pricing.

Component	Estimated monthly cost
EKS control plane	$73
Hub + supporting services (always-on node)	$60–122
Collector & policy worker nodes	$70–140
RDS PostgreSQL	$55–180
Networking & storage	~$37
Estimated total	**~$300–550/month**

The sections below break down each component with the assumptions behind these estimates.

Workload Profile

Repository activity

Tier	Repos	Commit frequency	Commits/month
Active	50	~2/day (main pushes + PRs)	3,000
Less active	150	~2/month	300

Tier	Repos	Commit frequency	Commits/month
Total	**200**		**3,300**

What triggers on each commit

When a commit lands on a tracked branch or PR, Lunar's hub receives a GitHub webhook and schedules the relevant collectors and policies.

- **Code collectors**: ~12 matching collectors fire per commit on average. Not all 20 match every repo — language-specific collectors that don't apply exit immediately.
- **CI collectors**: Run natively on the customer's CI runner. They consume zero hub compute and are excluded from this analysis.
- **Policies**: After collectors complete, ~30 policy checks evaluate the component. Policies are lightweight Python scripts that read pre-collected JSON data and produce pass/fail assertions. They complete in under one second each.

Collector weight distribution

Not all collectors are equal. On a typical commit:

Category	Collectors/commit	Avg duration	Examples
Fast-exit (wrong language, no data)	~5	<2s	Python collector on a Go repo
Lightweight (file scanning, API calls)	~4	<5s	README, CODEOWNERS, AI-use
Heavy (SBOM generation, linting, vuln scanning)	~3	30s–3min	Trivy, Syft, golangci-lint

Collector & Policy Pod Costs

Monthly pod invocations

With collector batching — grouping all matching collectors for a single component into one pod — the pod count drops significantly compared to scheduling individual pods per collector.

Workload	Pods/month	Avg duration	Total pod-hours
Collector batches (code hooks)	3,300	~60s avg	~55 hrs
Policy batches	3,300	~20s	~18 hrs
Cron collector batches (nightly)	~900	~45s	~11 hrs

Workload	Pods/month	Avg duration	Total pod-hours
Total	**~7,500**		**~84 hours**

Without batching, the same workload would produce ~150,000 individual pods/month. Batching — running 10–15 collectors or all 30 policy checks in a single pod — reduces pod count by approximately 95%. This is primarily an operational stability improvement: fewer pods means less Kubernetes API pressure, fewer scheduling decisions, and fewer image pulls. The raw compute time is similar either way.

Worker node sizing

Collector pods are ephemeral and bursty. During a busy hour, dozens of commits may arrive near-simultaneously. The cluster needs enough warm capacity to absorb these bursts without excessive queuing.

Configuration	Worker nodes	Monthly cost
Conservative (Spot instances, m5.large)	1–2 nodes	$70/month
Comfortable (On-demand, m5.large)	2 nodes	$140/month

The default pod resource spec from Lunar's K8s operator requests 250m CPU / 256Mi memory with limits of 1 CPU / 512Mi memory per snippet container. With batching, each pod runs the full suite for a component, so peak per-pod requirements are driven by the heaviest collector in the batch.

Hub Infrastructure

The hub is an always-on Go server that receives GitHub webhooks, schedules collector and policy runs, stores results in PostgreSQL, and serves the Grafana-based UI. It runs 24/7 alongside a small set of supporting services.

Hub services

Service	Purpose	Resource footprint
Hub server	gRPC + HTTP API, webhook handler, job queue	500m CPU, 1Gi memory
Grafana	Dashboards and UI	250m CPU, 512Mi memory

These services need to be running continuously and are counted here as one always-on node.

Configuration	Node type	Monthly cost
Conservative	t3.large (2 vCPU, 8Gi)	$60/month
Comfortable	t3.xlarge (4 vCPU, 16Gi)	$122/month

EKS control plane

The managed Kubernetes control plane is a fixed cost regardless of workload.

	Monthly cost
EKS cluster	$73/month

RDS PostgreSQL

Lunar's PostgreSQL database is the system of record for all collector output, policy results, component metadata, and the internal job queue. The schema includes ~20 tables with extensive JSONB storage, 30+ indexes, and materialized views that power the Grafana dashboards.

Estimated data volume (200 repos)

Table	Rows/month	Avg row size
Collector run records	~48,000	5–50 KB (JSONB blobs)
Snippet run tracking	~150,000	~500 bytes
Policy assertions	~100,000	~200 bytes
Merged component blobs	~3,300 (one per commit, per component)	10–100 KB

Recommended instances

Configuration	Instance	Storage	Monthly cost
Conservative	db.t4g.medium (2 vCPU, 4Gi, burstable)	50 GB gp3	~$55/month
Recommended	db.r6g.large (2 vCPU, 16Gi, dedicated memory)	100 GB gp3	~$180/month

The burstable `db.t4g.medium` is adequate for this workload under normal conditions. The `db.r6g.large` provides consistent performance for Grafana dashboard queries against materialized views and avoids CPU credit exhaustion during burst periods (e.g., when 50 repos push commits in the same hour and the hub is writing hundreds of collector results concurrently).

Automated backups and snapshots add approximately $5/month.

Networking & Storage

NAT Gateway

Pods in private subnets require a single-AZ NAT Gateway for outbound internet access — calling GitHub and third-party APIs and connecting to external services. With collector images cached in ECR (see below), the majority of NAT traffic is lightweight API calls.

	Monthly cost
NAT Gateway (single AZ)	~$35/month

A VPC endpoint for S3 (free gateway endpoint) further reduces data transfer through NAT.

S3 storage

Lunar uses S3 for two purposes: execution logs (retained for 30 days) and snippet resource archives (policy bundles, collector code packages zipped and stored for K8s pod init). At this scale, storage costs are negligible — under $2/month combined.

Container image registry

We recommend caching Lunar's collector images in the customer's own ECR registry. This avoids Docker Hub rate limits, eliminates image pull traffic through NAT, and keeps image pulls within the AWS network at no transfer cost. ECR storage for the full set of collector images is approximately 5 GB (~$1/month).

Cost Scaling

The table below shows how costs change as the deployment grows.

Scenario	Impact
400 repos (double)	Worker nodes +$50–100/month; RDS may need step-up
40 collectors (double)	Worker nodes +$80–140/month
10x commit velocity	Worker nodes need 5–8 instances: +$200–400/month
Add nightly cron collectors	+$20–40/month compute

The fixed infrastructure (EKS control plane, hub node, RDS) remains constant until the workload significantly exceeds the current tier. The variable cost — worker nodes for collector and policy pods — scales linearly with commit volume and collector count.

All estimates in this document assume a single-AZ deployment in a single region. Cross-AZ and cross-region data transfer costs are not included.

Architecture Notes

Hub: A single Go binary serving gRPC and HTTP, backed by PostgreSQL with the River job queue (Postgres-native, no external broker). The hub receives GitHub webhooks, resolves which collectors and policies apply to each component, and schedules execution.

Collectors: Bash scripts that run inside short-lived Kubernetes pods. They analyze source code, call external APIs, and write structured JSON to the hub. Collector images range from a lightweight 150 MB Alpine base (for file-scanning collectors) to ~800 MB (for collectors that bundle language toolchains like Go or Rust).

Policies: Python scripts that read the collected JSON data and produce pass/fail/skip assertions. All policies share a single lightweight base image. They are the least resource-intensive part of the system.

CI collectors: A subset of collectors that run natively on the customer's CI runners (GitHub Actions, etc.) rather than on hub infrastructure. These are excluded from this cost analysis as they consume no hub compute.

Batching: The upcoming batching improvement groups multiple collectors (or all policy checks) for the same component into a single pod. This reduces pod count by ~95% and significantly improves scheduling efficiency and cluster stability. The compute cost savings are modest (~$30–70/month) because the actual script execution time is unchanged — the savings come from eliminating per-pod scheduling overhead.
